

Functional Requirements

FR-1. File Selection

The system shall allow the user to select a BasicML program file from their computer through the GUI.

FR-2. File Validation

The system shall validate the selected program file by accepting properly formatted four-digit BasicML words, ignoring blank lines, rejecting malformed words, and enforcing the maximum program size.

FR-3. Program Loading

The system shall load a valid BasicML program into memory.

FR-4. File Error Recovery

The system shall display an error message for an invalid or malformed program file and allow the user to select another file without restarting the application.

FR-5. Machine Initialization

The system shall reset the machine state before execution of a newly loaded program begins.

FR-6. Start Execution

The system shall allow the user to start execution of a successfully loaded BasicML program.

FR-7. Stop Execution

The system shall allow the user to stop a BasicML program while it is executing.

FR-8. READ

The system shall request and accept a signed integer from -9999 through 9999 when a READ instruction is executed and store the value at the specified memory address.

FR-9. READ Validation

The system shall reject blank, non-integer, or out-of-range READ input and allow the user to correct and resubmit the input without restarting program execution.

FR-10. WRITE

The system shall display the value at the specified memory address in the Program Output area using four-digit magnitude formatting with a separate negative sign for negative values.

FR-11. LOAD

The system shall copy the value from the specified memory address into the accumulator.

FR-12. STORE

The system shall copy the accumulator value into the specified memory address, replacing its previous contents.

FR-13. ADD

The system shall add the value at the specified memory address to the accumulator.

FR-14. SUBTRACT

The system shall subtract the value at the specified memory address from the accumulator.

FR-15. MULTIPLY

The system shall multiply the accumulator by the value at the specified memory address.

FR-16. DIVIDE

The system shall divide the accumulator by the value at the specified memory address.

FR-17. BRANCH

The system shall continue execution at the specified memory address when a BRANCH instruction is executed.

FR-18. BRANCHNEG

The system shall continue execution at the specified memory address when a BRANCHNEG instruction is executed and the accumulator is negative.

FR-19. BRANCHZERO

The system shall continue execution at the specified memory address when a BRANCHZERO instruction is executed and the accumulator equals zero.

FR-20. HALT

The system shall stop program execution when a HALT instruction is executed.

FR-21. Program Output

The system shall preserve displayed program output after execution stops, completes, or encounters an error and clear the previous output when a new program is successfully loaded.

Non-Functional Requirements

NFR-1. Usability

The system shall provide clearly labeled GUI controls, prompts, input fields, and output areas so users can understand their purpose.

NFR-2. Modularity

The system shall keep user-interface code separate from the simulator's data model and business logic using separate classes or modules.

NFR-3. Responsiveness

The system shall remain responsive during program execution so the user can interact with GUI controls, including the Stop control.

